

Yes — Task 1 stays exactly as it is. For **Task 2 only**, we add a 10-second per-question timer.

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0
        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        self.num_a = 0
        self.num_b = 0
        self.operator = "+"

        # Task 2: 10-second countdown timer
        self.time_limit = 10
        self.time_remaining = self.time_limit
        self.last_update_time = pygame.time.get_ticks()

        box_w, box_h = 130, 44
        self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
        self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_card = pygame.font.SysFont(None, 56)
        self.font_btn = pygame.font.SysFont(None, 24)

        self.generate_new_card()

    def generate_new_card(self):
        self.num_a = random.randint(3, 15)
        self.num_b = random.randint(2, 12)
        self.operator = random.choice(["+", "-", "*"])
        if self.operator == "-" and self.num_a < self.num_b:
            self.num_a, self.num_b = self.num_b, self.num_a

        self.input_box.clear()

        # Task 2: reset timer for every new question
        self.time_remaining = self.time_limit
        self.last_update_time = pygame.time.get_ticks()

    def compute_expected_answer(self):
```

```

    if self.operator == "+":
        return self.num_a + self.num_b
    elif self.operator == "-":
        return self.num_a - self.num_b
    elif self.operator == "*":
        return self.num_a * self.num_b

def submit_answer(self):
    val_str = self.input_box.text.strip()
    if not val_str or val_str == "-":
        self.feedback_msg = "Type an answer first!"
        self.feedback_color = (240, 175, 40)
        return

    user_answer = int(val_str)
    expected = self.compute_expected_answer()
    self.total_attempts += 1

    if user_answer == expected:
        self.score += 1
        self.feedback_msg = f"CORRECT! {self.num_a} {self.operator} {self.num_b} = {expected}"
        self.feedback_color = (80, 230, 110)
        self.generate_new_card()
    else:
        self.feedback_msg = f"WRONG! Expected {expected}."
        self.feedback_color = (240, 75, 75)
        self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
        self.submit_answer()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_answer()

def update(self):
    # Task 2: update the countdown timer
    current_time = pygame.time.get_ticks()
    elapsed_seconds = (current_time - self.last_update_time) / 1000.0
    self.last_update_time = current_time

    if self.time_remaining > 0:
        self.time_remaining -= elapsed_seconds

    if self.time_remaining <= 0:
        self.time_remaining = 0
        self.total_attempts += 1
        self.feedback_msg = "TIME'S UP!"
        self.feedback_color = (240, 75, 75)
        self.generate_new_card()

```

```

def render(self, screen):
    screen.fill((25, 29, 37))

    title_surf = self.font_title.render("Math Flashcards Arena", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 18))

    score_surf = self.font_hud.render(f"Score: {self.score} / {self.total_attempts}", True, (25, 29, 37))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 58))

    card_rect = pygame.Rect(self.width // 2 - 130, 95, 260, 110)
    pygame.draw.rect(screen, (240, 242, 245), card_rect, border_radius=12)
    pygame.draw.rect(screen, (85, 120, 175), card_rect, width=3, border_radius=12)

    card_str = f"{self.num_a} {self.operator} {self.num_b}"
    card_surf = self.font_card.render(card_str, True, (25, 30, 42))
    screen.blit(card_surf, (card_rect.centerx - card_surf.get_width() // 2, card_rect.centery))

    # Task 2: visible countdown timer bar below the card
    timer_x = self.width // 2 - 130
    timer_y = 215
    timer_width = 260
    timer_height = 10

    timer_ratio = max(0, self.time_remaining / self.time_limit)
    current_timer_width = int(timer_width * timer_ratio)

    pygame.draw.rect(
        screen,
        (70, 75, 85),
        pygame.Rect(timer_x, timer_y, timer_width, timer_height),
        border_radius=5
    )

    pygame.draw.rect(
        screen,
        (80, 190, 110),
        pygame.Rect(timer_x, timer_y, current_timer_width, timer_height),
        border_radius=5
    )

    timer_text = self.font_btn.render(
        f"Time: {max(0, self.time_remaining):.1f}s",
        True,
        (220, 225, 235)
    )
    screen.blit(
        timer_text,
        (
            self.width // 2 - timer_text.get_width() // 2,
            timer_y + timer_height + 5
        )
    )

```

```

    )

    self.input_box.render(screen)

    pygame.draw.rect(screen, (45, 140, 80), self.submit_btn, border_radius=6)
    pygame.draw.rect(screen, (215, 225, 220), self.submit_btn, width=2, border_radius=6)
    btn_txt = self.font_btn.render("SUBMIT", True, (255, 255, 255))
    screen.blit(btn_txt, (self.submit_btn.centerx - btn_txt.get_width() // 2, self.submit

    msg_surf = self.font_hud.render(self.feedback_msg, True, self.feedback_color)
    screen.blit(msg_surf, (self.width // 2 - msg_surf.get_width() // 2, 295))

```

What changed for Task 2

- Added `time_limit = 10` and `time_remaining`.
- Timer resets to **10 seconds** inside `generate_new_card()`, so every new question gets a fresh timer.
- `update()` calculates elapsed time using `pygame.time.get_ticks()`.
- When the timer reaches `0`:
 - `total_attempts` increases by 1.
 - "TIME'S UP!" is displayed.
 - A new card is generated.
 - The timer resets to 10 seconds.
- Added a **visible countdown bar and seconds display** below the card.
- Task 1's `compute_expected_answer()` is **unchanged**.
- No streak multiplier, division, or Task 4 functionality was added.



Task 1 and Task 2 are completed and tested successfully.

Now implement **ONLY** Task 3 from the README.

Implement consecutive correct streak multipliers.

Requirements:

- Maintain a consecutive correct-answer streak.
- Increase the streak after every correct answer.
- At 3 consecutive correct answers, use a 2x multiplier.
- At 5 consecutive correct answers, use a 3x multiplier.

Show more ▾

Yes. We'll preserve **Task 1** and **Task 2** exactly, and add only the streak/multiplier logic for Task 3.

Task 3 changes

- Added `self.streak = 0`.
- Added `get_multiplier()`:
 - 0–2 correct → 1×
 - 3–4 correct → 2×
 - 5+ correct → 3×
- Correct answer increments the streak and awards the corresponding points.
- Wrong answer resets streak to `0`.
- Timeout resets streak to `0`.
- Added a visible `Streak: X | Multiplier: Xx` display.
- Timer and Task 1 arithmetic remain intact.

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0
        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        self.num_a = 0
        self.num_b = 0
        self.operator = "+"

        # Task 2: 10-second countdown timer
        self.time_limit = 10
        self.time_remaining = self.time_limit
        self.last_update_time = pygame.time.get_ticks()

        # Task 3: consecutive correct-answer streak
        self.streak = 0

        box_w, box_h = 130, 44
```

```
self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

self.font_title = pygame.font.SysFont(None, 38)
self.font_hud = pygame.font.SysFont(None, 26)
self.font_card = pygame.font.SysFont(None, 56)
self.font_btn = pygame.font.SysFont(None, 24)

self.generate_new_card()

def generate_new_card(self):
    self.num_a = random.randint(3, 15)
    self.num_b = random.randint(2, 12)
    self.operator = random.choice(["+", "-", "*"])
    if self.operator == "-" and self.num_a < self.num_b:
        self.num_a, self.num_b = self.num_b, self.num_a

    self.input_box.clear()

    # Task 2: reset timer for every new question
    self.time_remaining = self.time_limit
    self.last_update_time = pygame.time.get_ticks()

def compute_expected_answer(self):
    if self.operator == "+":
        return self.num_a + self.num_b
    elif self.operator == "-":
        return self.num_a - self.num_b
    elif self.operator == "*":
        return self.num_a * self.num_b

def get_multiplier(self):
    # Task 3: determine multiplier from consecutive correct answers
    if self.streak >= 5:
        return 3
    elif self.streak >= 3:
        return 2
    else:
        return 1

def submit_answer(self):
    val_str = self.input_box.text.strip()
    if not val_str or val_str == "-":
        self.feedback_msg = "Type an answer first!"
        self.feedback_color = (240, 175, 40)
        return

    user_answer = int(val_str)
    expected = self.compute_expected_answer()
    self.total_attempts += 1

    if user_answer == expected:
```

```

# Task 3: increase consecutive correct streak
self.streak += 1

multiplier = self.get_multiplier()
points_earned = multiplier

self.score += points_earned

self.feedback_msg = (
    f"CORRECT! {self.num_a} {self.operator} {self.num_b} = "
    f"{expected} (+{points_earned})"
)
self.feedback_color = (80, 230, 110)
self.generate_new_card()

else:
    # Task 3: incorrect answer resets streak
    self.streak = 0

    self.feedback_msg = f"WRONG! Expected {expected}."
    self.feedback_color = (240, 75, 75)
    self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
        self.submit_answer()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_answer()

def update(self):
    # Task 2: update the countdown timer
    current_time = pygame.time.get_ticks()
    elapsed_seconds = (current_time - self.last_update_time) / 1000.0
    self.last_update_time = current_time

    if self.time_remaining > 0:
        self.time_remaining -= elapsed_seconds

    if self.time_remaining <= 0:
        self.time_remaining = 0
        self.total_attempts += 1

    # Task 3: timeout resets streak
    self.streak = 0

    self.feedback_msg = "TIME'S UP!"
    self.feedback_color = (240, 75, 75)
    self.generate_new_card()

```

```
def render(self, screen):
    screen.fill((25, 29, 37))

    title_surf = self.font_title.render(
        "Math Flashcards Arena",
        True,
        (245, 245, 245)
    )
    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,
            18
        )
    )

    score_surf = self.font_hud.render(
        f"Score: {self.score} / {self.total_attempts}",
        True,
        (255, 220, 80)
    )
    screen.blit(
        score_surf,
        (
            self.width // 2 - score_surf.get_width() // 2,
            58
        )
    )

    # Task 3: display current streak and multiplier
    multiplier = self.get_multiplier()
    streak_surf = self.font_hud.render(
        f"Streak: {self.streak} | Multiplier: {multiplier}x",
        True,
        (220, 225, 235)
    )
    screen.blit(
        streak_surf,
        (
            self.width // 2 - streak_surf.get_width() // 2,
            82
        )
    )

    card_rect = pygame.Rect(
        self.width // 2 - 130,
        105,
        260,
        110
    )
    pygame.draw.rect(
        screen,
```

```

        (240, 242, 245),
        card_rect,
        border_radius=12
    )
    pygame.draw.rect(
        screen,
        (85, 120, 175),
        card_rect,
        width=3,
        border_radius=12
    )

    card_str = f"{self.num_a} {self.operator} {self.num_b}"
    card_surf = self.font_card.render(
        card_str,
        True,
        (25, 30, 42)
    )
    screen.blit(
        card_surf,
        (
            card_rect.centerx - card_surf.get_width() // 2,
            card_rect.centery - card_surf.get_height() // 2
        )
    )

    # Task 2: visible countdown timer bar below the card
    timer_x = self.width // 2 - 130
    timer_y = 225
    timer_width = 260
    timer_height = 10

    timer_ratio = max(
        0,
        self.time_remaining / self.time_limit
    )
    current_timer_width = int(
        timer_width * timer_ratio
    )

    pygame.draw.rect(
        screen,
        (70, 75, 85),
        pygame.Rect(
            timer_x,
            timer_y,
            timer_width,
            timer_height
        ),
        border_radius=5
    )

```

```
pygame.draw.rect(
    screen,
    (80, 190, 110),
    pygame.Rect(
        timer_x,
        timer_y,
        current_timer_width,
        timer_height
    ),
    border_radius=5
)

timer_text = self.font_btn.render(
    f"Time: {max(0, self.time_remaining):.1f}s",
    True,
    (220, 225, 235)
)
screen.blit(
    timer_text,
    (
        self.width // 2 - timer_text.get_width() // 2,
        timer_y + timer_height + 5
    )
)

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (45, 140, 80),
    self.submit_btn,
    border_radius=6
)

pygame.draw.rect(
    screen,
    (215, 225, 220),
    self.submit_btn,
    width=2,
    border_radius=6
)

btn_txt = self.font_btn.render(
    "SUBMIT",
    True,
    (255, 255, 255)
)
screen.blit(
    btn_txt,
    (
        self.submit_btn.centerx - btn_txt.get_width() // 2,
        self.submit_btn.centery - btn_txt.get_height() // 2
    )
)
```

```
)

msg_surf = self.font_hud.render(
    self.feedback_msg,
    True,
    self.feedback_color
)
screen.blit(
    msg_surf,
    (
        self.width // 2 - msg_surf.get_width() // 2,
        295
    )
)
```

Streak behavior

Consecutive correct	Multiplier	Points for next correct answer
1	1×	+1
2	1×	+1
3	2×	+2
4	2×	+2
5	3×	+3
6+	3×	+3

A wrong answer or timeout immediately resets the streak to 0 and multiplier to 1×.

One small layout adjustment was necessary: the streak display sits between the score and card, and the timer bar was moved slightly down so the new display doesn't overlap the card.



Task 1, Task 2 and Task 3 are completed and tested successfully.

Now implement ONLY Task 4 from the README.

Add integer division support.

Requirements:

- Add "/" as an operator in generate_new_card().
- Division must always produce a whole-number result with no remainder

Show more ▾

Absolutely. For **Task 4 only**, I'll add integer division while preserving the existing Task 1–3 behavior.

The division generation uses a non-zero divisor and constructs `num_a` as a multiple of `num_b`, so every `/` question has an exact whole-number answer.

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0
        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        self.num_a = 0
        self.num_b = 0
        self.operator = "+"

        # Task 2: 10-second countdown timer
        self.time_limit = 10
        self.time_remaining = self.time_limit
        self.last_update_time = pygame.time.get_ticks()

        # Task 3: consecutive correct-answer streak
        self.streak = 0

        box_w, box_h = 130, 44
        self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
        self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 26)
```

```
self.font_card = pygame.font.SysFont(None, 56)
self.font_btn = pygame.font.SysFont(None, 24)

self.generate_new_card()

def generate_new_card(self):
    self.num_a = random.randint(3, 15)
    self.num_b = random.randint(2, 12)
    self.operator = random.choice(["+", "-", "*", "/"])

    # Task 4: generate division questions with no remainder
    if self.operator == "/":
        self.num_b = random.randint(2, 12)
        quotient = random.randint(2, 12)
        self.num_a = self.num_b * quotient

    elif self.operator == "-" and self.num_a < self.num_b:
        self.num_a, self.num_b = self.num_b, self.num_a

    self.input_box.clear()

    # Task 2: reset timer for every new question
    self.time_remaining = self.time_limit
    self.last_update_time = pygame.time.get_ticks()

def compute_expected_answer(self):
    if self.operator == "+":
        return self.num_a + self.num_b
    elif self.operator == "-":
        return self.num_a - self.num_b
    elif self.operator == "*":
        return self.num_a * self.num_b
    elif self.operator == "/":
        # Task 4: integer division with an exact result
        return self.num_a // self.num_b

def get_multiplier(self):
    # Task 3: determine multiplier from consecutive correct answers
    if self.streak >= 5:
        return 3
    elif self.streak >= 3:
        return 2
    else:
        return 1

def submit_answer(self):
    val_str = self.input_box.text.strip()
    if not val_str or val_str == "-":
        self.feedback_msg = "Type an answer first!"
        self.feedback_color = (240, 175, 40)
        return
```

```
user_answer = int(val_str)
expected = self.compute_expected_answer()
self.total_attempts += 1

if user_answer == expected:
    # Task 3: increase consecutive correct streak
    self.streak += 1

    multiplier = self.get_multiplier()
    points_earned = multiplier

    self.score += points_earned

    self.feedback_msg = (
        f"CORRECT! {self.num_a} {self.operator} {self.num_b} = "
        f"{expected} (+{points_earned})"
    )
    self.feedback_color = (80, 230, 110)
    self.generate_new_card()

else:
    # Task 3: incorrect answer resets streak
    self.streak = 0

    self.feedback_msg = f"WRONG! Expected {expected}."
    self.feedback_color = (240, 75, 75)
    self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
        self.submit_answer()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_answer()

def update(self):
    # Task 2: update the countdown timer
    current_time = pygame.time.get_ticks()
    elapsed_seconds = (current_time - self.last_update_time) / 1000.0
    self.last_update_time = current_time

    if self.time_remaining > 0:
        self.time_remaining -= elapsed_seconds

    if self.time_remaining <= 0:
        self.time_remaining = 0
        self.total_attempts += 1

    # Task 3: timeout resets streak
    self.streak = 0
```

```
        self.feedback_msg = "TIME'S UP!"
        self.feedback_color = (240, 75, 75)
        self.generate_new_card()

def render(self, screen):
    screen.fill((25, 29, 37))

    title_surf = self.font_title.render(
        "Math Flashcards Arena",
        True,
        (245, 245, 245)
    )
    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,
            18
        )
    )

    score_surf = self.font_hud.render(
        f"Score: {self.score} / {self.total_attempts}",
        True,
        (255, 220, 80)
    )
    screen.blit(
        score_surf,
        (
            self.width // 2 - score_surf.get_width() // 2,
            58
        )
    )

    # Task 3: display current streak and multiplier
    multiplier = self.get_multiplier()
    streak_surf = self.font_hud.render(
        f"Streak: {self.streak} | Multiplier: {multiplier}x",
        True,
        (220, 225, 235)
    )
    screen.blit(
        streak_surf,
        (
            self.width // 2 - streak_surf.get_width() // 2,
            82
        )
    )

    card_rect = pygame.Rect(
        self.width // 2 - 130,
        105,
```

```
        260,  
        110  
    )  
    pygame.draw.rect(  
        screen,  
        (240, 242, 245),  
        card_rect,  
        border_radius=12  
    )  
    pygame.draw.rect(  
        screen,  
        (85, 120, 175),  
        card_rect,  
        width=3,  
        border_radius=12  
    )  
  
    card_str = f"{self.num_a} {self.operator} {self.num_b}"  
    card_surf = self.font_card.render(  
        card_str,  
        True,  
        (25, 30, 42)  
    )  
    screen.blit(  
        card_surf,  
        (  
            card_rect.centerx - card_surf.get_width() // 2,  
            card_rect.centery - card_surf.get_height() // 2  
        )  
    )  
  
    # Task 2: visible countdown timer bar below the card  
    timer_x = self.width // 2 - 130  
    timer_y = 225  
    timer_width = 260  
    timer_height = 10  
  
    timer_ratio = max(  
        0,  
        self.time_remaining / self.time_limit  
    )  
    current_timer_width = int(  
        timer_width * timer_ratio  
    )  
  
    pygame.draw.rect(  
        screen,  
        (70, 75, 85),  
        pygame.Rect(  
            timer_x,  
            timer_y,  
            timer_width,
```

```
        timer_height
    ),
    border_radius=5
)

pygame.draw.rect(
    screen,
    (80, 190, 110),
    pygame.Rect(
        timer_x,
        timer_y,
        current_timer_width,
        timer_height
    ),
    border_radius=5
)

timer_text = self.font_btn.render(
    f"Time: {max(0, self.time_remaining):.1f}s",
    True,
    (220, 225, 235)
)
screen.blit(
    timer_text,
    (
        self.width // 2 - timer_text.get_width() // 2,
        timer_y + timer_height + 5
    )
)

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (45, 140, 80),
    self.submit_btn,
    border_radius=6
)

pygame.draw.rect(
    screen,
    (215, 225, 220),
    self.submit_btn,
    width=2,
    border_radius=6
)

btn_txt = self.font_btn.render(
    "SUBMIT",
    True,
    (255, 255, 255)
)
screen.blit(
```

```
        btn_txt,\n        (\n            self.submit_btn.centerx - btn_txt.get_width() // 2,\n            self.submit_btn.centery - btn_txt.get_height() // 2\n        )\n    )\n\n    msg_surf = self.font_hud.render(\n        self.feedback_msg,\n        True,\n        self.feedback_color\n    )\n    screen.blit(\n        msg_surf,\n        (\n            self.width // 2 - msg_surf.get_width() // 2,\n            295\n        )\n    )
```

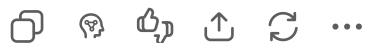
Task 4 addition

For `/`, the code generates questions like:

- $12 / 3 = 4$
- $30 / 5 = 6$
- $72 / 8 = 9$

Because `num_a` is generated as `num_b * quotient`, **there is always no remainder**, and `num_b` is always between 2 and 12, so division by zero cannot occur.

Everything from **Tasks 1, 2, and 3** is preserved.



ChatGPT can make mistakes. Check important info.